Demo Script

For judges during live presentation (target time: 4-5 minutes).

---

0. Setup (30 sec — before judging)

- Ensure `make local-up` is running (or AWS deployed).
- Have the GitHub repo open on a secondary screen.

---

1. Hook & Problem (0:00-0:45)

> *"Today we show how OpenCV 5 on AWS Graviton via the COOL library gives better fps-per-dollar than stock OpenCV on x86."*
>
> - *Point at the dashboard's top-bar: instance type, COOL version, live FPS, hourly cost, and fps/$.*

---

2. Live Demo (0:45-2:30)

2.1 Dashboard walkthrough

> *"On the left, live video with bounding boxes. On the right, metric cards and charts."*
>
> - *Hover over FPS card: explain it's end-to-end pipeline latency (capture → inference → draw → metric emit).*
> - *Show latency percentiles (p50/p95/p99) from the CloudWatch custom metrics.*
> - *Point to the cost card: derived from instance hourly price and measured FPS.*

2.2 Toggle between COOL and Stock (if stretch feature enabled)

> *"With one click we can switch the pipeline to stock OpenCV on the same instance type to see the delta."*
>
> - *Show fps/$ drop when switching to stock.*

2.3 Download benchmark

> *"Click **Download Benchmark** to get the per-frame CSV we just collected."*
>
> - *Show a few rows: timestamp, latency_ms, inference_ms, detections.*
> - *Mention the same CSV is uploaded to S3 in the AWS deployment.*

---

3. Architecture Reveal (2:30-3:15)

> *"Here's how it's wired."*
>
> - *Open `docs/architecture.mmd` (or the rendered Mermaid on `/architecture` page).*
> - *Walk through the layers:*
>   1. *Camera → RTSP/file source*
>   2. *EC2 c8g.xlarge running COOL AMI (Arm KleidiCV-accelerated OpenCV 5)*
>   3. *Pipeline: YOLO11n (DNN) + ALIKED/LightGlue (Features2D)*
>   4. *Telemetry: per-frame CSV → S3; custom metrics → CloudWatch*
>   5. *Dashboard: Next.js 16 App Router, Tailwind v4, Server-Sent Events*
>   6. *Judge's browser: live metrics + video*

---

4. Reproducibility (3:15-3:45)

> *"One command reproduces this locally."*
>
> - *Show `make local-up` → localhost:3000.*
> - *Note the Docker compose runs a stub pipeline + LocalStack so no AWS credentials are needed for development.*
> - *For AWS, `make deploy-infra` provisions the stack; `make benchmark` runs the comparison.*

---

5. Responsible Use & Close (3:45-4:15)

> *"No PII is collected; video never leaves the EC2 instance. We used CC0 sample video (Big Buck Bunny)."*
>
> - *The system is a benchmark, not a production CV pipeline.*
> - *Invite the judge to scan the QR code (or visit the repo) to see the code, docs, and reproduction steps.*

---

6. Q&A Ready

- Have the `technical-report.mdx` and `reproduction-guide.md` open for deep dives.
- Be ready to discuss:
  - Why COOL requires AWS Marketplace AMI.
  - Which OpenCV 5 features are in use (DNN engine, ALIKED, LightGlue).
  - How the fps/$ metric is calculated (FPS ÷ hourly instance price).
  - What we'd change with more time (multi-region failover, Kinesis streams, automated CI).